

# SCALE: Tackling Communication Bottlenecks in Confidential Distributed Machine Learning

Joongun Park<sup>†</sup>, Yongqin Wang<sup>‡</sup>, Huan Xu<sup>†</sup>, Hanjiang Wu<sup>†</sup>, Mengyuan Li<sup>‡</sup>, Tushar Krishna<sup>†</sup>

<sup>†</sup> Georgia Institute of Technology, <sup>‡</sup> University of Southern California

jpark3234@gatech.edu, yongqin@usc.edu, hxu398@gatech.edu, hwu419@gatech.edu, mli49061@usc.edu, tushar@ece.gatech.edu

**Abstract**—Machine Learning (ML) has become a cornerstone in numerous applications, creating the need for secure and efficient distributed ML frameworks. However, maintaining data privacy in these systems poses significant challenges, particularly in distributed environments where user data and model parameters must frequently be transmitted between GPUs. Confidential GPU computing technologies, such as NVIDIA’s Confidential Computing (CC) mode, offer hardware-based enterprise solutions designed to protect ML workloads in untrusted environments (e.g., public clouds). These technologies leverage heterogeneous systems that combine Confidential Virtual Machines (CVMs) with GPU-based Trusted Execution Environments (TEEs). Nevertheless, confidential computing introduces considerable performance overhead due to its complex heterogeneous architecture and the high-throughput data flows required across TEE security boundaries. For example, encrypted communication occurs both between CVMs and GPU TEEs, and among multiple GPU TEEs, resulting in significant latency compared to native PCIe or high-speed interconnects such as NVLink. Our extensive evaluation shows that these overheads become particularly severe during collective communication operations, which suffer from encryption-induced delays that negatively impact end-to-end training performance.

To address this, we propose a co-encryption design that leverages underutilized GPU resources, optimizes encryption and authentication, and introduces a communication algorithm tailored for confidential settings. We evaluate our design using real ML workloads and execution traces collected from four HGX H100/H200 clusters. While CC mode was not available on current NVIDIA software stacks, we incorporate encryption-aware modeling based on hardware specifications to estimate secure communication overheads. Our results demonstrate a 40–70% reduction in communication-related security costs.

## I. INTRODUCTION

Machine learning (ML) systems are now fully integrated into everyday services, from healthcare and finance to code assistants and chatbots. Increasingly, these systems are not only trained on public datasets, but also fine-tuned or personalized using sensitive user data, such as private documents, customer interactions, source code, and medical records. When such training pipelines run on cloud infrastructure, the cloud provider gains full visibility into both the training process and the underlying raw data, posing fundamental privacy risks.

To address these concerns, major cloud providers such as Azure and Google Cloud have begun offering confidential computing services based on NVIDIA’s Confidential GPU technology [7], [11], [25], [33]. These systems extend Trusted Execution Environment (TEE) protections into each GPU

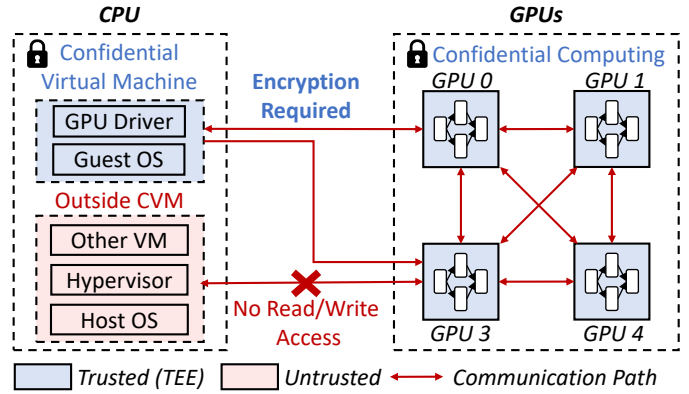


Fig. 1: Overview of Confidential GPU Computing System.

(e.g., NVIDIA H100, B200, and RTX pro 6000), enabling secure ML training in shared, multi-tenant cloud environments. However, protecting a single GPU is only the first hurdle. Modern ML workloads, especially those that involve large-scale models, heavily depend on distributed computations that span multiple GPUs, demanding significant data throughput. As shown in Figure 1, the CPU executes inside a Confidential Virtual Machine (CVM), while each GPU runs in Confidential Computing (CC) mode. In this setting, data must remain encrypted at rest, protected during computation, and kept confidential during transit between devices. This security requirement introduces a critical performance bottleneck: *protected inter-device communication*.

Specifically, this bottleneck arises primarily due to the limited throughput of current GPU hardware encryption engines, typically capped around PCIe speeds ( $\sim 64$  GB/s, unidirectional). This throughput lags significantly behind the native capabilities of high-speed GPU interconnects like NVLink ( $\sim 450$  GB/s, unidirectional) [7], [27], [30]. Even state-of-the-art FPGA-based encryption engines achieve only up to 5 GB/s of encrypted bandwidth [9], [39], making it infeasible to sustain encrypted communication near NVLink speeds. In addition, security metadata (e.g., authentication tags, initialization vectors) and encryption latency inflate communication time and bandwidth consumption. These issues are especially pronounced in collective communication operations, such as AllReduce and AllGather, which dominate training time in distributed ML. When secure communication delays synchronization, GPU compute units stall, leading to idle hardware and extended training duration. Thus, end-to-end encryption

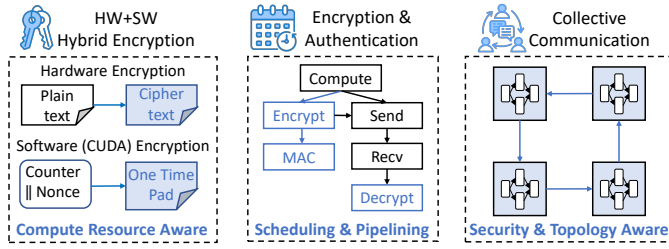


Fig. 2: Overview of SCALE system.

on distributed ML pipelines, while critical for confidentiality, also imposes severe performance overhead.

To address the performance challenges introduced by secure communication, we introduce SCALE<sup>1</sup>, a unified framework that combines software–hardware co-encryption with secure-aware collective optimizations. Figure 2 illustrates the high-level design objectives. (1) At the heart of SCALE lies a co-encryption mechanism that exploits otherwise idle GPU resources—compute units or dedicated copy engines—during communication phases. By partitioning the encryption workload between the built-in hardware engine and these spare resources, SCALE significantly reduces end-to-end latency without weakening TEE level confidentiality guarantees. (2) To further reduce overhead, SCALE precomputes One-Time Pads (OTPs) during idle periods and defers MAC verification until checkpointing. This decouples encryption and authentication from the critical path, enabling better overlap and reducing bandwidth overhead. (3) Complementing co-encryption, SCALE applies topology-aware optimizations (e.g., ring, tree) for secure collectives (e.g., AllReduce, AllGather), adapting algorithms to the constraints of encrypted communication. Together, these techniques form a cohesive design that is compatible with existing multi-GPU training frameworks, enabling high throughput while preserving strong privacy guarantees.

The contributions are summarized as follows:

- We present SCALE, the first end-to-end study on accelerating encrypted inter-device communication in confidential GPU multi-node training, revealing communication as the primary bottleneck in existing TEE-based solutions.
- SCALE introduces a software–hardware co-encryption mechanism that leverages idle GPU compute and copy engines to parallelize encryption work, reducing link latency without weakening security.
- SCALE designs secure-aware collective communication algorithms, applying topology-specific scheduling, metadata reduction, and encryption-transmission overlap to accelerate encrypted AllReduce, AllGather, and similar operations.
- SCALE is evaluated on large-scale LLM workloads in 4 x HGX H100/H200 GPU clusters, combining real-system measurements with simulation to demonstrate scalability and effectiveness. The evaluation results demonstrate a 40–70% overhead reduction caused by confidential GPU computing.

<sup>1</sup>Securing Collectives for Accelerated distributed Learning with Encryption

## A. Distributed Machine Learning

Distributed training accelerates machine learning by spreading computation across multiple devices, such as GPUs or accelerators, making it essential for large-scale models that exceed a single device’s capacity [23]. Data and model parallelism [35], [39] are the keys for distributed training. Data parallelism splits data across devices with each training a model copy, while model parallelism divides the model itself. Though this improves computational efficiency, it incurs communication overhead from synchronization.

**Collective Communications.** Collective communication is a core component of distributed ML, responsible for coordinating and synchronizing data exchanges among multiple devices during training. Key operations include all-reduce, broadcast, AllGather, and ReduceScatter. For instance, AllReduce operation is used by data parallelization to update weight gradients after the backward pass. NVIDIA Collective Communications Library (NCCL) is a widely used framework for optimizing these collective communication operations on GPUs [32]. NCCL is designed to optimally leverage high-speed interconnects like NVLink, PCIe, and InfiniBand, achieving near-peak performance in distributed environments. By handling low-level details of device communication, NCCL allows ML frameworks such as PyTorch and TensorFlow to implement distributed training with high performance and scalability.

## B. Confidential GPUs and Trusted Execution Environments

In NVIDIA’s confidential GPUs, hardware-level encryption and secure memory isolation mechanisms protect data throughout processing and communication. Each GPU includes a dedicated copy engine that manages encrypted DMA transfers between GPUs or between CPU and GPU memory. These data transfers utilize bounce buffers [22] in device-local memory to temporarily hold ciphertext, which is then copied into secure regions for decryption. This design ensures that plaintext data never exits the GPU’s trusted boundary. Consequently, all inter-GPU or CPU-GPU communications over NVLink or PCIe remain encrypted, preserving confidentiality against adversaries who might physically inspect the interconnects. NVIDIA GPUs employ AES-GCM-256 (256 key bit length) [8], a symmetric authenticated encryption algorithm that guarantees both confidentiality and integrity. AES-GCM can also detect replay attacks by tracking each unique nonce and counter pair for every encryption operation. Due to the symmetric nature of AES-GCM, both the sender and receiver must share a common secret key to encrypt and decrypt the communication. Management of this key is one of the key components of NVIDIA CC architecture.

**Key Management.** Key management in NVIDIA’s CC architecture involves secure initialization and attestation. GPUs establish a secure key storage within their trusted boundary, which stores essential cryptographic materials like session keys and initialization vectors (IVs). Through an attestation mechanism, GPUs securely derive communication keys from

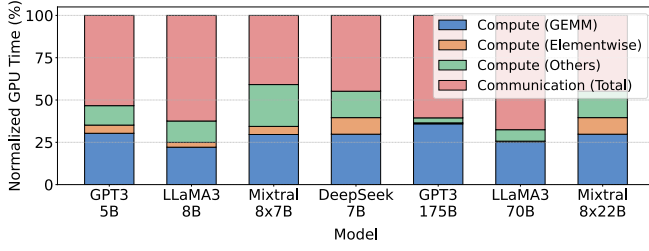


Fig. 3: Normalized breakdown of GPU operation time across large-scale ML models on H200 HGX cluster.

a master key provisioned during setup. Once initialized, keys remain confined to GPU hardware, significantly reducing the risk of exposure to other system components.

### III. MOTIVATION: UNDERSTANDING THE PERFORMANCE BOTTLENECKS IN DISTRIBUTED CONFIDENTIAL ML

Confidential Computing (CC) provides confidential ML by enforcing hardware-level isolation and encrypting memory. However, extending CC to large-scale multi-GPU training introduces non-trivial performance costs, particularly when encrypting traffic over high-speed but untrusted interconnects. This overhead is amplified by the collective operations needed for various parallelism strategies (e.g., data, model, expert). We identify two key challenges that make secure distributed training difficult and motivate new optimization opportunities.

#### A. Challenges in Securing Distributed ML

**Challenge 1: Encryption Bandwidth Bottleneck.** Secure ML training demands encrypted and authenticated communication across GPUs; however, current hardware encryption modules (e.g., AES-GCM) deliver limited throughput (reported to be 64 GB/s), drastically lower than native high-speed interconnects like NVLink, which offer bandwidth up to 450 GB/s [2], [3]. This discrepancy creates a substantial bottleneck, severely impeding collective communication performance. Since computational tasks often depend on data/gradient synchronization via collectives, encryption and authentication overhead can cascade into a system-wide performance degradation, substantially lengthening training times [11], [23].

Figure 3 shows our profiling result across different models (CPU runtime is not significant). The result shows that communication is taking 44.8% to 88.6%, which is a significant portion of total operation time. In addition, profiling results from distributed ML training workloads highlight that practical bandwidth achieved (390 GB/s, unidirection) during training far exceeds hardware encryption capabilities, emphasizing this performance mismatch. Consequently, optimizing secure communication is paramount to achieving scalable and efficient confidential ML.

**Challenge 2: Diverse Communication Patterns.** Another challenge in secure distributed ML training lies in the vast optimization search space, driven by the diverse communication patterns required by different parallelization strategies. This diversity makes performance tuning under CC particularly difficult. Different parallelization strategies such as data parallel (DP), tensor/sequence parallel (TP/SP), pipeline parallel (PP), and expert parallel (EP) induces distinct communication

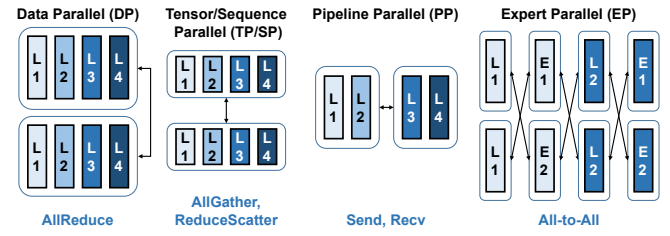


Fig. 4: Parallelization Strategies and Communication

(PP), and expert parallel (EP) induces distinct communication patterns, including AllReduce, AllGather, Send/Recv, and All-to-All, based on how the models and data are distributed across devices, as illustrated in Figure 4. To optimize performance, existing frameworks adopt different collective algorithms based on data size, operation type, and interconnect topology, typically balancing latency and bandwidth in non-secure environments.

However, in confidential computing (CC) settings, all communication must be encrypted and authenticated, which substantially alters performance dynamics. Each communication pattern interacts differently with encryption primitives, leading to varying overheads and making it harder to design secure and efficient collectives [23].

#### B. Exploiting GPU Underutilization for Protection

Despite challenges arising from cryptographic overheads and a diverse optimization search space, our evaluation across various ML tasks reveals significant opportunities for achieving both high-performance and secure distributed ML training by leveraging underutilized GPU resources. Specifically, GPUs tend to under-utilize their computational resources during collective communication phases, especially when there is limited opportunity to overlap communication with computation. Table I shows our profiling results of Streaming Multiprocessors (SMs) utilization and GPU idle time across various large language models (LLMs) and GPU scales. The results indicate that, during communication-heavy periods, less than 25% of SMs are active on average—even for large-scale training runs involving 32 x H200 GPUs. This trend is consistent across different model sizes and parallelization configurations.

In addition to communication, GPUs may remain idle while awaiting control decisions or synchronization signals from the CPU, such as during kernel launches or coordination barriers. The communication pattern also varies depending on the model architecture and its parallelism strategy. For example, MoE models such as Mixtral utilize All-to-All communication, while dense transformer models rely more heavily on AllReduce or ReduceScatter. Importantly, increased parallelism often reduces per-GPU message sizes, which limits the number of threads participating in communication kernels. As a result, only a subset of available SMs are engaged during communication, leaving others idle and underutilized.

Consequently, even in large models like GPT3 175B [10] or LLaMA3 70B [12], a substantial portion of GPU cycles re-

TABLE I: Profiling results of LLMs. Average GPU idle time and Streaming Multiprocessor (SM) utilization during collective communication types: AllReduce (AR), AllGather (AG), ReduceScatter (RS), and All-to-All (A2A).

| Model        | Param.          | GPU Idle | Avg. SM Utilization during Communication |
|--------------|-----------------|----------|------------------------------------------|
| GPT-3        | 5B (8xH200)     | 8.1%     | AR=0.76%, AG=18.2%, RS=18.2%             |
|              | 175B (32xH200)  | 28.3%    | AR=1.51%, AG=1.51%, RS=1.51%             |
| LLaMA-3      | 8B (8xH200)     | 1.9%     | AR=18.2%                                 |
|              | 70B (16xH200)   | 1.0%     | AR=1.51%, AG=1.51%, RS=1.51%             |
| Mixtral [17] | 8x7B (8xH200)   | 13.5%    | A2A=24.2%                                |
|              | 8x22B (32xH200) | 2.2%     | AR=18.2%, AG=18.2%, RS=18.2%, A2A=6.0%   |

mains underutilized. This underutilization presents a valuable opportunity: we can launch a lightweight, independent GPU kernel such as encryption kernel on the otherwise idle SMs without interfering with the main training pipeline. The auxiliary kernel must be issued under the same security authority to preserve the trust boundary. Doing so can alleviate pressure on dedicated encryption engines or software stacks, improving overall system throughput while preserving confidentiality.

#### IV. SCALE DESIGN DETAILS

This paper introduces SCALE, designed to optimize the performance in confidential distributed training.

**SCALE Overview.** SCALE tackles the challenges discussed in Section III through three synergistic approaches. First, we propose a GPU-optimized software-hardware co-encryption algorithm that opportunistically leverages underutilized GPU compute units to accelerate encrypted data transfers (Section IV-B). Second, we introduce a resource-aware OTP pre-computation and MAC verification, offloading latency-critical encryption and authentication operations from the critical communication path. This technique eliminates secure metadata transfers and significantly reduces overhead (Section IV-C). Third, and crucially, we found that existing NCCL collective communication algorithms needs to incorporate security-awareness, systematically analyzing various secure collective strategies through extensive empirical evaluation. Our approach identifies and mitigates encryption-induced bottlenecks (Section IV-E), enhancing performance while ensuring robust security. Through these strategies, we provide a practical and scalable solution for confidential distributed ML training.

##### A. Threat Model

SCALE considers an identical threat model as the one defined by the NVIDIA CC framework [7]. In this model, the adversary is assumed to have full control over the software stack and have physical access to the cloud server hosting the GPU CC environment. This includes the ability to run any privileged instructions, such as running custom virtual machines and kernel modules, as well as inspect hardware interfaces like interconnect buses. As a result, the attacker can observe or tamper with any data outside the TEE boundaries (CPU/GPU TEE), including traffic traversing PCIe, NVLink, and DRAM, through software or physical attacks. This threat model is illustrated in Figure 1.

Given this adversarial model, we place trust in the hardware integrity of the CPU and GPU packages, including the High Bandwidth Memory (HBM) that is tightly integrated into the GPU package. We also include all code executing within the CPU and GPU TEEs in the trusted computing base (TCB). Since the user supplies this code, the responsibility of ensuring its correctness lies with the user. Consistent with the NVIDIA CC model and other commercially available TEEs, we exclude denial-of-service (DoS) attacks and side-channel attacks from our threat model. Providing robust defense mechanisms against all forms of side channels remains a complex and largely unsolved problem that lies beyond the guarantees of current commercial TEE solutions.

##### B. Accelerated AES Encryption

**AES-GCM (Galois/Counter Mode).** Figure 5 illustrates the internal structure of AES-GCM, which consists of AES in Counter Mode (AES-CTR) to encrypt data and a Galois Hash (GHASH) to authenticate it. AES-CTR generates a keystream from a secret key and nonce, which is XORed with plaintext for encryption. GHASH computes a message authentication code (MAC) on the ciphertext and metadata to detect tampering and prevent replay attacks.

**SW Encryption Performance.** Figure 6 illustrates the performance of our CUDA-optimized software encryption, demonstrating the effectiveness of GPU parallelism in accelerating AES-CTR (OTP computation) and GHASH MAC (GMAC) computations. H100 and H200 GPUs exhibit comparable throughput, reaching up to 486 GB/s for AES-CTR and 908 GB/s for GMAC when configured with 1024 threads per block. This similarity arises from their shared Hopper compute architecture, and the observed performance indicates that both operations are compute-bound rather than limited by memory or interconnect bandwidth.

**Precomputing OTP.** To further reduce encryption overhead, SCALE precomputes OTPs during idle GPU periods, leveraging the message-independence property of AES-CTR encryption (Figure 7 (b),(c)). These OTPs are securely stored in GPU’s on-package HBM (global memory), enabling encryption to be performed as a fast XOR operation (up to 1350 GB/s) during communication. As a result, AES-CTR can be effectively decoupled from critical path in communication [26], [38], [41]. This enables SCALE to hide encryption latency behind communication and compute phases, improving performance without compromising security. Similarly, authentication can also be decoupled during the secure communication as detailed in following section IV-D.

**Resource Profiling and Optimization Opportunities.** According to NCCL’s design, each communication channel is mapped to a single CUDA block, which exclusively occupies one SM [4], [5], [43]. Consequently and as we observed in III-B only a limited number of SMs are used for communication kernels, while the remaining SMs are left available for other operations such as computation. While many techniques have been proposed to improve compute-communication over-

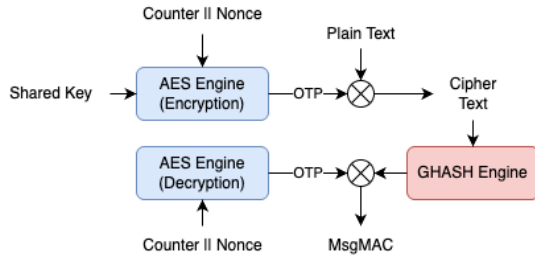


Fig. 5: AES-GCM Encryption and Decryption Mechanism

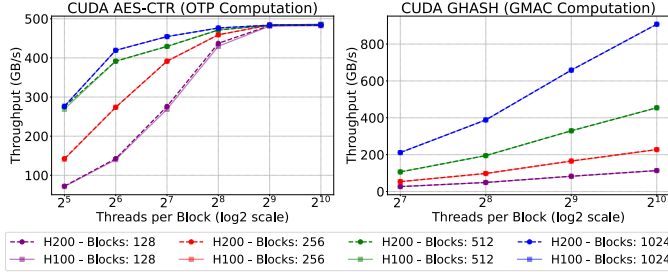


Fig. 6: AES-CTR and GHASH throughput scaling with GPU thread and block configurations

lap, we found that many compute kernels remain stalled [12], [17], [39] due to data dependencies rather than SM contention. This limits the effectiveness of overlap and leaves substantial compute capacity idle during communication.

**Co-encryption Strategy.** These findings motivate our co-encryption design, which assigns encryption tasks to underutilized GPU compute units during communication-heavy periods.

### C. HW+SW Hybrid Encryption and Authentication

SCALE introduces a hybrid software-hardware encryption mechanism that opportunistically leverages idle GPU SMs to offload parts of the AES-GCM pipeline in software. The standard AES-GCM consists of AES-CTR encryption for confidentiality and GHASH-based authentication for integrity.

#### Algorithm 1 Hybrid Encryption Selection

```

1: procedure HYBRIDENCRYPT(Message)
2:   if  $\text{size}(\text{Message}) \leq \text{size}(\text{OTP}_{\text{avail}})$  then
3:      $\text{CipherText} \leftarrow \text{SW\_Encrypt\_XORing}(\text{Message}, K)$ 
4:   else
5:     // Once the hardware path is selected, no fallback to software path
6:     // within the same collective communication.
7:      $\text{CipherText} \leftarrow \text{HW\_Encrypt}(\text{Message}, K)$ 
8:   end if
9:   return CipherText
10: end procedure

```

**Hybrid Encryption.** At runtime, SCALE dynamically selects the encryption path for each message using a lightweight hybrid policy (Algorithm 1). The policy considers message size, availability of OTPs, and current GPU utilization. If a fresh OTP of sufficient size is available, the message is encrypted via the software fast path. Otherwise, the system falls back to hardware encryption while idle SMs are computing OTPs for next SW encryption. This runtime path is embedded in

#### Algorithm 2 Utilizing Idle SMs for GHASH and AES-CTR

```

1: procedure UTILIZESM( $\text{CipherText}, \text{OTP}_{\text{avail}}, \text{SM}_{\text{util}}$ )
2:   // Opportunistically utilize idle SM capacity
3:   while  $\text{SM}_{\text{util}} < \text{Util}_{\text{threshold}}$  do
4:     // Prioritize integrity verification
5:     if  $\text{size}(\text{CipherText}) > 0$  then
6:       // Chained GMAC accumulation
7:        $\text{GMAC} \leftarrow \text{CUDA\_GHASH}(\text{CipherText}, \text{GMAC}, \text{SM}_{\text{util}})$ 
8:     else if  $\text{size}(\text{OTP}_{\text{avail}}) < \text{Max\_OTP\_Size}$  then
9:       // Precompute OTP until it reaches max size
10:       $\text{target} \leftarrow \text{Max\_OTP\_Size} - \text{size}(\text{OTP}_{\text{avail}})$ 
11:       $\text{OTP}_{\text{gen}} \leftarrow \text{CUDA\_AES\_CTR}(\text{key}, \text{target}, \text{SM}_{\text{util}})$ 
12:       $\text{OTP}_{\text{avail}} \leftarrow \text{Increase\_OTP\_Avail}(\text{OTP}_{\text{gen}})$ 
13:     else
14:       break
15:     end if
16:   end while
17:   return DONE
18: end procedure

```

the message metadata, allowing receivers to authenticate and decrypt accordingly. The approach ensures that encryption overhead is minimized while maintaining TCB isolation and full confidentiality.

**Counter and OTP Management.** Each GPU maintains an independent, monotonically increasing counter space for every GPU pair, ensuring that each ciphertext/OTP is always derived from a unique (key, counter) pair. During communication between two GPUs, the corresponding counter is incremented for each OTP generation used in encryption or decryption. The counter value is never decreased or reset (except for during nonce rekeying), eliminating the possibility of counter reuse. AES kernels on GPU use these counters to generate OTPs for secure communication with each peer. Our NCCL backend consumes the pre-computed OTPs when performing communication with GPUs. In our system, three factors ensure counters and OTPs are never reused: 1) per-GPU pair monotonic counters, 2) a single-producer-single-consumer OTP model, and 3) reliable NVLink delivery. Each GPU pair maintains an independent, strictly increasing counter space that guarantees uniqueness across different collective algorithm choices. The single-producer-single-consumer design ensures only one AES kernel reads counters and generates OTPs, avoiding contention and locks. NVLink provides in-order delivery with link-layer error detection and retry mechanisms [1], simplifying counter synchronization without requiring additional software-level reordering logic.

**Kernel Scheduling.** Modern GPUs support concurrent kernel execution across independent CUDA streams. In SCALE, we exploit GPU idle periods during collective communications to perform cryptographic operations in parallel with NVLink transmissions. This allows our cryptographic kernel to execute minimizing interfering with critical compute workloads such as matrix multiplications during training.

When the GPU has no available idle resources or the OTP pool runs low, software encryption is skipped and the collective transparently falls back to the hardware AES-GCM path, while the crypto kernel focuses on replenishing OTPs for the next round instead of attempting to switch encryption

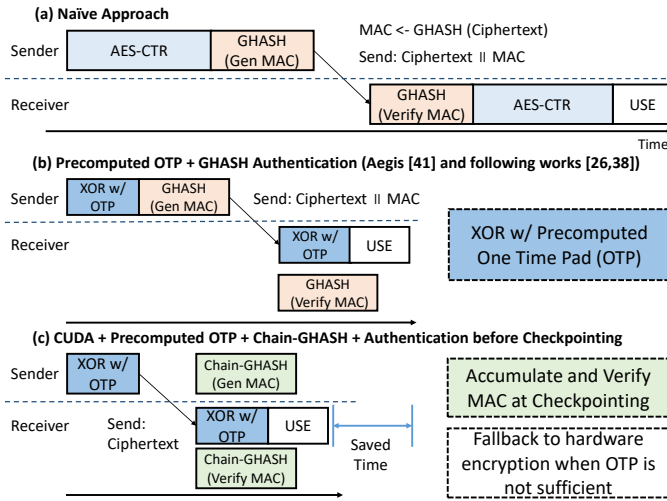


Fig. 7: Precomputed OTP w/ Parallel GHASH verification

modes mid-collective. Within the crypto kernel, operations are ordered such that XOR is executed first, followed by GHASH and AES-CTR pre-computation, prioritizing critical-path encryption completes first while future OTPs and authentication tags are prepared in the background. All OTP and GMAC data produced by the crypto kernel reside in GPU HBM within the confidential-computing boundary.

**Secure Key Handling and Multi-node Extensions.** As NVIDIA’s confidential GPU support is currently limited to single-node deployments, we propose a secure multi-node key management scheme based on a cluster of CVMs. Each GPU first performs remote attestation with a designated CVM in its node to verify TEE integrity. Upon successful attestation, the CVMs coordinate secure key establishment across nodes using the Elliptic Curve Diffie-Hellman protocol [14]. This enables per-link key generation without requiring direct GPU-to-GPU communication between nodes. We assume that this key exchange process is secure and is part of the TCB.

#### D. Optimized Authentication via Deferred MAC

SCALE adopts a multi-chaining authentication [48], which addresses the inherent serialization of MAC generation and verification in AES-GCM. In standard AES-GCM-256, authentication is strictly sequential (Figure 8(a)), creating a bottleneck for parallel processing. Multi-chaining design relaxes this constraint by dividing the MAC computation across multiple independent chains, enabling significantly greater parallelism without compromising security. Leveraging this approach, SCALE achieves parallel authentication while maintaining the same security guarantees as conventional AES-GCM-256. Additionally, SCALE further optimizes integrity verification by deferring MAC checks to predefined synchronization points, such as model checkpoints or epoch boundaries [31], [47], rather than performing them inline during communication (Figure 8(c)). The correctness of this deferred MAC validation is grounded in the concept of lazy integrity, which has been widely adopted in prior work on multi-GPU

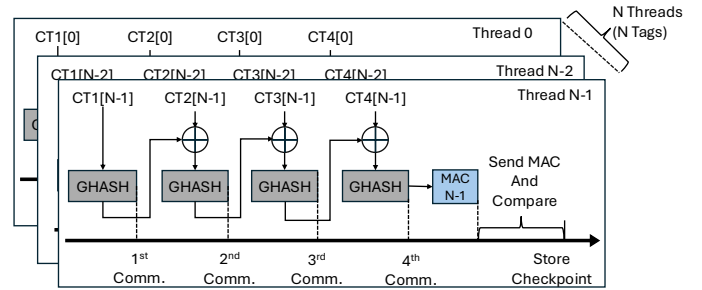


Fig. 8: Illustration for multi-chaining MAC Generation and Deferred Verification for Checkpoint Storage.

and multi-CPU systems [26], [37], [50]. To further illustrate, we show Figure 8 to explain how multi-chaining MAC generation works in combination with deferred MAC verification. In the figure, there are 4 CT (cipher texts) data communicated between the neighboring GPUs before checkpointing. Each CT data is divided into N blocks and we adopt the multi-chaining aforementioned for that. Instead of directly sending the MAC for verification, we continue the chaining till the checkpoint is reached. Then, we send the MAC tags generated among all the threads over to the receiver for batched validation.

This strategy provides multiple benefits. First, it eliminates the need to complete GHASH computations before transmission, thereby reducing latency introduced by encryption. Second, it removes MAC verification from the receiver’s critical path, enabling faster processing of incoming encrypted messages without blocking on integrity checks. Third, by deferring verification, per-message authentication tags are no longer transmitted during communication. This reduces metadata overhead per message. In AES-GCM-256, the authentication tag is 16 bytes, which can introduce up to 25% overhead for small messages. Deferral shifts this bandwidth cost to infrequent synchronization points (e.g., checkpoints), instead of paying it during every collective operation.

Along with OTP precomputation, SCALE schedules deferred MAC computations during periods when GPU resources are idle or underutilized (Algorithm 2). MAC generation is prioritized over OTP precomputation to reduce memory pressure caused by retaining ciphertexts that await verification. **Authentication at Checkpointing.** While deferring MAC verification introduces a delay in detecting potential tampering, this tradeoff is acceptable under our threat model where all GPUs operate within a TEE and communication is fully encrypted (see Section IV-A). Importantly, most ML pipelines perform periodic checkpointing, which provides a natural boundary for integrity checks.

SCALE verifies aggregated MACs at the beginning of each checkpoint. If any tampering is detected, the corresponding checkpoint is simply discarded and training can resume from a previous clean state. Earlier checkpoints stay immutable with different filename (e.g., checkpoint\_#), ensuring a clean recovery point is always available. This design ensures end-to-end integrity without requiring immediate per-message authentication, allowing the system to maintain both correctness

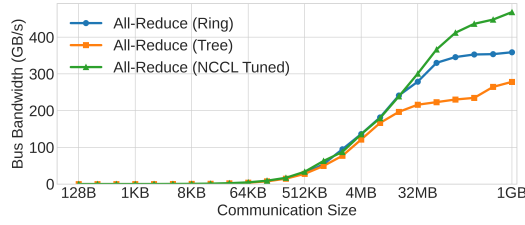


Fig. 9: Performance of NCCL All-Reduce algorithms.

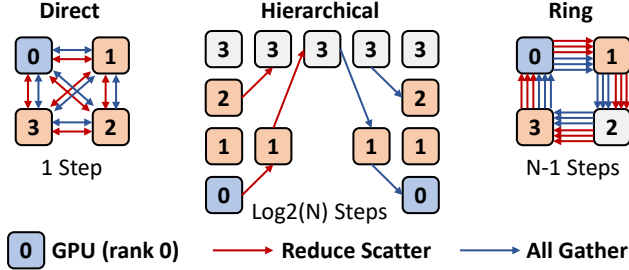


Fig. 10: AllReduce variants (AllGather + ReduceScatter)

and efficiency.

Thus, by aligning security checks with checkpointing, SCALE preserves the guarantees of authenticated encryption while removing verification from the critical path—resulting in lower latency and reduced bandwidth pressure during training.

#### E. Security-Aware Collective Communication

**Inefficiency of NCCL under Encryption.** NVIDIA NCCL includes multiple variants of collective algorithms—such as Ring, Tree, and CollNet—which have different communication patterns and degrees of parallelism. Since the performance of each algorithm depends on factors such as message size, system topology, and GPU count, it is necessary to identify the most performant algorithm. *libNCCL-tuner* builds a cost model based on the latency and bandwidth of each algorithm to select the most suitable one according to message size. Figure 9 shows that *libNCCL-tuner* can significantly improve AllReduce bandwidth, as measured on one 8-GPU H100 node. However, *libNCCL-tuner* does not consider security-related performance factors such as encryption bandwidth and pre-computed OTP size. The following section gives a detailed case study on OTP consumption under different algorithms.

**Analyzing Collective Communication and OTP.** Figure 10 illustrates representative communication patterns of three common AllReduce variants. Direct propagates the entire message to all peers in a single communication step, while Ring pipelines the workload over  $N-1$  fixed-size transfers. Hierarchical collectives reduce the number of communication rounds to logarithmic in  $N$  by progressively aggregating and disseminating data. These differences in communication structure directly impact key management and encryption overhead.

Figure 11 illustrates how the available precomputed OTPs in the buffer evolve over time for Direct, Ring, and Tree collective algorithms during secure FSDP training of the GPT3 5B

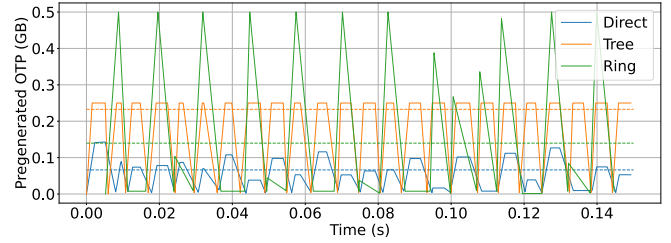


Fig. 11: OTP occupancy during GPT3 5B FSDP training

TABLE II: Best Practices for Collectives (RS, AG, AR) and P2P (A2A) Communication with sufficient OTP: NCCL vs. Security-Aware

| Method | Data Size     | RS   | AG   | AR     | A2A    |
|--------|---------------|------|------|--------|--------|
| NCCL   | 128 B–512 KB  | Tree | Tree | Tree   | Direct |
|        | 512 KB–2 MB   | Ring | Ring | Ring   | Direct |
|        | $\geq 2$ MB   | NVLS | NVLS | NVLS   | Direct |
| SCALE  | 128 B–4 MB    | Tree | Tree | Tree   | Direct |
|        | 4–128 MB      | Ring | Tree | Hybrid | Direct |
|        | $\geq 128$ MB | Ring | Ring | Ring   | Direct |

NVLS denotes an NCCL-specific hardware-assisted collective path. SCALE does not rely on NVLS, as NVSwitch is not trusted under our threat model.

model. For fair comparison, we fixed the total OTP buffer size per GPU to 1 GB across all settings. This buffer is partitioned among the communication peers of each GPU, which varies depending on the collective algorithm:  $N-1$  peers for Direct, 2 for Ring, and  $\log_2 N$  for Tree, where  $N$  is the total number of GPUs. As a result, the per-peer OTP allocation decreases with the number of concurrent communication paths required by each algorithm.

On average, Direct maintains the smallest OTP buffer occupancy at 0.07 GB, followed by Ring at 0.14 GB, and Tree maintains the highest level at 0.23 GB. These trends reflect how each algorithm’s communication structure affects the OTP generation and consumption dynamics. Tree-based collectives exhibit a gradual OTP usage pattern and sustain higher buffer levels over time. In contrast, Direct tends to deplete its OTP buffer sharply, frequently exhausting available OTPs and triggering fallback to hardware-based encryption. Ring initially sustains a relatively high OTP buffer level. However, the buffer level decreases steadily and eventually falls below Tree’s. Ring continuously consumes OTPs throughout execution due to its hop-by-hop communication pattern and ends up consuming the largest total amount of OTP among the three algorithms.

These trends can shift depending on factors such as message size, pre-computed OTP size, parallelism degree, and interconnect bandwidth. This variability calls for a dynamic collective algorithm selection policy. To address this, we introduce *SCALE-tuner*, which dynamically selects the optimal collective algorithm based on a security-aware cost model.

#### F. Extending NCCL Profiling for Secure Communication

NVIDIA’s NCCL runtime estimates the latency of every supported topology  $a$  (e.g., Ring, Tree, NVLS) and transfer

protocol  $p$  before each collective, then launches the pair with the minimum predicted time. The baseline model is

$$T(a, p, n) = \underbrace{\alpha(a, p) \cdot \text{launches}(a)}_{\text{start-up}} + \underbrace{\beta(a, p) n}_{\text{steady state}},$$

where  $n$  is the message size. A *Tree* topology pipes all stages inside a single kernel launch ( $\text{launches} = 1$ ), whereas a *Ring* serialises its  $2(P-1)$  logical hops ( $\text{launches} = 2(P-1)$ ). The start-up term  $\alpha$  aggregates (i) a fixed launch cost, (ii) one intra-GPU link latency per hop and (iii) one inter-node latency when the communicator spans multiple nodes. The per-byte term  $\beta$  is the reciprocal of the effective bandwidth left after NCCL applies the protocol's limitations.

**Security-aware Parameters.** To capture the impact of hybrid encryption on collective performance, we extend NCCL's latency model with security-aware parameters.

We first define the software encryption ratio  $r$  as:

$$r = \frac{\min(n_{\text{perGPU}}, \text{OTP}_{\text{avail}})}{n_{\text{perGPU}}}$$

Using  $r$ , the encryption-induced overheads are modeled as:

$$\Delta\alpha = r \cdot \alpha_{\text{SW}} + (1-r) \cdot \alpha_{\text{HW}}, \quad \Delta\beta = \frac{1}{r \cdot B_{\text{SW}} + (1-r) \cdot B_{\text{HW}}}$$

where  $\alpha_{\text{SW}}$  and  $\alpha_{\text{HW}}$  denote the startup latency of software and hardware encryption, respectively, and  $B_{\text{SW}}$ ,  $B_{\text{HW}}$  are their corresponding encryption throughput.

**Observed Crossover Shift.** Table III contrasts the best-performing All-Reduce strategies selected by the plain-text *NCCL-tuner* and the security-aware *SCALE-tuner* on an 8-GPU H100 node. With authenticated-encryption enabled, the *SCALE-tuner* delays the Tree to Ring crossover until 4 MB because Tree's single-launch schedule amortises the constant MAC/XOR overhead and consumes fewer OTP per byte. For medium messages (4–128 MB) it employs a *hybrid* plan: the reduce-scatter phase runs as a Ring while the all-gather phase remains Tree-based, reflecting that encryption inflates the cost of aggregation (many-to-one) more than fan-out (one-to-many). Arithmetic on ciphertext is infeasible, so NVLS is disabled; for messages  $\geq 128$  MB the tuner therefore reverts to a pure Ring, which spreads the per-byte encryption cost uniformly across all active links.

#### G. Compatibility with Existing CC Framework

NVIDIA's open-GPU-kernel-modules (580.82.07) define internal registry keys (e.g., *RmNvlinkEncryption* in *src/nvidia/interface/nvrm\_registry.h*) that control NVLink encryption, confirming that a driver-level switch already exists, though no public API currently exposes it. To fully enable SCALE's hybrid encryption, future GPUs would need a secure driver interface permitting controlled runtime modification of this flag. Since the GPU driver is TCB under the CC model, such an interface would not expand the trusted boundary.

For correctness, each message carries a one-bit tag indicating whether software or hardware encryption was used. Finally, *SCALE-tuner* augments *libNCCL-tuner*'s cost model with encryption parameters, enabling security-aware collective selection.

## V. SECURITY ANALYSIS

Our threat model is identical to NVIDIA CC and adversaries in our threat model are not able to access data within CPU and GPU packages. However, they are able to observe traffic over (1) memory bus between CPU and DRAM, (2) PCIe, and (3) NVLink. SCALE does not modify the communication protocols over either the memory bus or PCIe. Therefore, it inherits the same confidentiality and integrity guarantees as NVIDIA CC for these channels. Since SCALE introduces modifications only to the communication protocol over NVLink, the following subsections focus on analyzing SCALE's security guarantees for preserving confidentiality and integrity over NVLink.

**Confidentiality.** All NVLink traffic in SCALE is encrypted using AES-CTR, which corresponds to the encryption component of AES-GCM. SCALE ensures that nonce/counter pairs are never reused and that all encryption keys are securely stored within the protected keystore. Consequently, SCALE provides the same level of confidentiality as AES-CTR-256, identical to the guarantees offered by NVIDIA CC.

**Integrity and MAC Verification Deferral.** SCALE preserves the original GHASH function and adheres to established best practices, ensuring that any traffic modification is detected with the same probability as AES-GCM-256. To improve performance, SCALE defers intermediate MAC verifications and performs them before checkpoint. If a MAC mismatch is detected, training immediately halts and all optimizer and model states are rolled back to the last authenticated checkpoint, guaranteeing correctness and preventing further propagation of corrupted gradients or weights. This deferral impacts only the latency of attack detection, without compromising correctness or integrity, as any attack will still be detected before model deployment. While more frequent checkpoints or periodic MAC verification could further bound wasted computation in case of an attack, they incur higher runtime overhead during normal operation—representing a tunable trade-off between detection granularity and performance.

**Replay Protection.** SCALE offers replay protection by associating each message with a unique nonce and monotonic counter, consistent with AES-GCM. Even when MAC verification is deferred, the encryption layer enforces strict nonce uniqueness to prevent replays or duplicate packet acceptance.

**Hybrid Encryption Safety.** The dynamic selection between hardware and software encryption paths does not weaken the security model, as both mechanisms use the same AES-GCM-256 encryption standard. SCALE tags each message with the encryption path, and any failure to verify the MAC will be caught at the verification points. On each GPU, one SCALE kernel is the sole producer of OTPs and NCCL is the sole consumer. This model does not incur race conditions and requires no locks or additional synchronization overhead. An attacker might flip the tag to make a hardware-encrypted packet look like a software-encrypted one (or vice versa). This still fails, because the two paths have different keys and counters. So that it fails to verify MAC before checkpoint.

**Side-Channel Attacks.** Both NVIDIA CC and SCALE follow the same threat model [29], which explicitly excludes side-channel attacks such as timing, cache, or power analysis. Consequently, an adversary observing timing or traffic may still infer meta-information, such as packet size, whether encryption is handled in hardware or software, approximate OTP utilization, communication volume, or collective-communication patterns. In SCALE, however, the relevant signals (ciphertext length, workload intensity, and the ordering of collective operations) are already visible in baseline CC’s NCCL and therefore do not broaden the attack surface. These channels are orthogonal to the confidentiality and integrity guarantees targeted by NVIDIA CC and this work; comprehensive mitigation is outside our scope and left to future research. Moreover, both CPU and GPU sides run constant-time, data-oblivious AES-CTR/GHASH code, which minimizes possible side-channel against the encryption procedure itself.

## VI. DISCUSSION

**Generality and Applicability.** SCALE has been evaluated on diverse distributed workloads including LLM, CNN, and MoE training, showing that its hybrid co-encryption adapts well to collective patterns such as AllReduce, AllGather, ReduceScatter, and All-to-All. It is designed to operate below the application layer and remains compatible with frameworks such as NCCL. The deferred-MAC mechanism particularly benefits training and fine-tuning, where periodic checkpoints provide natural verification boundaries and allow overlap between encryption and integrity checks. Inference workloads, being latency-sensitive and checkpoint-free, perform immediate verification, though they still benefit from SCALE’s hybrid path selection for lower communication latency.

**Beyond ML Workloads.** SCALE’s approach generalizes to confidential workloads with heavy GPU–GPU communication and partial idle time, such as HPC and graph analytics, where it can leverage idle resources to hide encryption latency and improve throughput.

**Applicability to other Confidential Architectures.** The principle of hybrid co-encryption and idle-resource utilization can extend to other TEE-based confidential computing architectures facing similar interconnect bottlenecks. SCALE does not target homomorphic encryption, which protects data even during computation under a distinct threat model, but its core ideas may be conceptually applicable to other confidential computing systems under different threat models.

## VII. EVALUATION

### A. Evaluation Methodology with Real-Execution Traces

**Methodology.** We evaluate our study using a modified version of ASTRA-sim [49], a distributed ML simulator, together with Chakra execution traces [40]. We rely on simulation because the latest versions of NCCL and the NVIDIA driver [28] do not yet support confidential computing in multi-GPU setups, which makes it difficult to directly measure encryption overheads in large-scale distributed training. Accordingly, we modified and calibrated the ASTRA-Sim to simulate secure

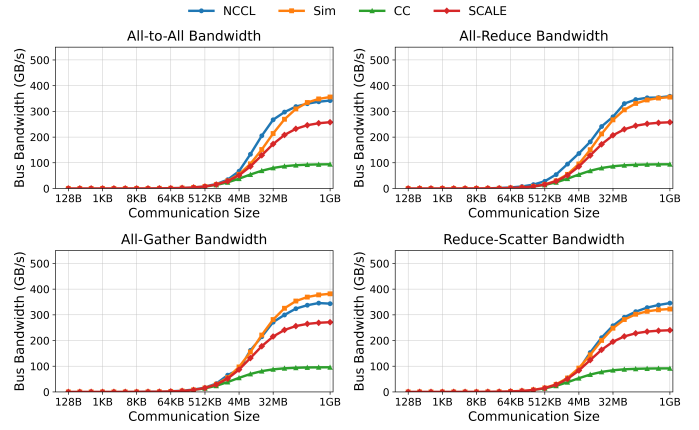


Fig. 12: Collective Communication: Simulation vs. NCCL Benchmark.

TABLE III: Models used for secure collective communication simulation. Megatron-style models involve hybrid parallelism strategies that result in diverse collective patterns.

| Model            | Param.                             | Micro batch / Batch |
|------------------|------------------------------------|---------------------|
| GPT3             | 5B (FSDP), 175B (TP/SP=32)         | 1/128, 1/32         |
| LLaMA3           | 8B (TP/SP=8), 70B (TP/SP=16)       | 1/128, 1/32         |
| Mixtral          | 8x7B (EP=8), 8x22B (TP/SP=4, EP=8) | 1/128, 1/32         |
| DeepSeek-MoE [6] | 7B (EP=8)                          | 1/128               |
| ResNet-50 [18]   | 25.6M (DP=8)                       | 1024/1024           |

communication across various parallelization settings and model architectures. To achieve high-fidelity simulation, all simulation inputs (execution traces, compute timestamp, communication schedules, and encryption penalties) are directly measured on a NVIDIA HGX H100 and H200 cluster. We execute large-scale models on those clusters and extracted traces from actual training executions. We use PyTorch 2.6.0 [15], NVIDIA NeMo 24.07 [19], Megatron-LM 0.10.0 [39], CUDA 12.4 and NCCL 2.21 in our software stack.

**Simulation Modeling and Validation.** We model the current CC architecture and its associated threat model for simulation. In distributed ML, execution consists of two main components: computation and communication. For computation, since data within GPU HBM remains unencrypted in CC mode, compute-bound operations (e.g., GEMM, GeLU, Attention) run at nearly native speed [7]. These operations are replayed in simulation using measured execution times from actual hardware. For communication, we first calibrated simulation accuracy against real NCCL performance, as shown in Figure 12. The comparison (NCCL vs. Sim) spans multiple collective types, algorithms, and chunk sizes, achieving an average geometric mean error of 17% across all settings. We assume hardware encryption bandwidths of 4 GB/s for CPU–GPU and 64 GB/s for GPU–GPU communication, consistent with NVIDIA’s announcement and prior work [2], [27], [29], [42]. Software encryption costs are benchmarked for AES-CTR, XOR, and GHASH on both H100 and H200 GPUs (Figure 6), and these overheads are injected into each send/receive operation to model secure communication.

TABLE IV: Communication and Encryption Counts per GPU per Training Epoch

| Model               | AR     | AG     | RS     | A2A  | SW Encryption | HW Encryption |
|---------------------|--------|--------|--------|------|---------------|---------------|
| GPT3 5B             | 9      | 392    | 200    | 0    | 18,978        | 15,182        |
| LLaMA3 8B           | 16,897 | 0      | 0      | 0    | 1,024,753     | 867,711       |
| Mixtral 8x7B        | 32     | 0      | 0      | 2048 | 211,887       | 250,449       |
| DeepSeek-MoE        | 18     | 23     | 23     | 1728 | 237,538       | 154,126       |
| GPT3 175B           | 130    | 18,528 | 12,320 | 0    | 5,500,265     | 2,214,519     |
| LLaMA3 70B          | 66     | 15,456 | 10,304 | 0    | 1,906,252     | 1,200,788     |
| Mixtral 8x22B       | 243    | 1,134  | 905    | 896  | 226,578       | 115,526       |
| Mixtral 8x7B (LoRA) | 1059   | 1792   | 1024   | 1024 | 397,552       | 104,432       |
| ResNet-50           | 1      | 0      | 0      | 1024 | 112           | 0             |

**Workload.** Table III summarizes the models and configurations used for trace collection and simulation. For each model, we simulate both baseline (unencrypted) and secure (encrypted) training environments. The batch size is chosen based on model size and GPU memory capacity to ensure feasible execution. Specifically, for 8B-scale models, we use a batch size of 128 and a micro-batch size of 1, while for larger models (70B+), we reduce the global batch size to 32. For ResNet-50, both batch sizes are set to 1024.

### B. SCALE for Encrypted Collective Communications

Table IV reports the number of collective communication operations and encryption invocations per GPU per training epoch across several representative ML models. The type and volume of communication operations vary significantly depending on the model architecture and parallelization strategy. Dense models such as LLaMA3 and DeepSeek exhibit a mix of AllReduce, AllGather, and ReduceScatter, reflecting typical combinations of data and tensor parallelism. In contrast, MoE-based models such as Mixtral and DeepSeek adopt extensive All-to-All communication due to expert routing across GPUs. They motivate the design of SCALE, which actively exploits hybrid co-encryption and per-collectives optimization. A key observation is that SCALE can effectively substitute HW encryption with SW encryption which dominates the total encryption workload across many models. It helps reduce pressure on hardware encryption engines—particularly when collective communication throughput exceeds the limited bandwidth of hardware AES-GCM engines. Thus, a higher proportion of SW encryption suggests that a system effectively offloads encryption to software, overlapping it with communication or computation phases.

They also motivate the design of SCALE, which actively exploits software-based encryption and per-collectives optimization. By harnessing idle GPU resources to precompute OTPs and by tailoring collectives to the secure communication pattern, SCALE delivers improved efficiency in secure distributed ML training environments.

**Performance Speedup Achieved by SCALE.** As shown in Figure 12, SCALE consistently outperforms the baseline CC implementation across communication sizes from 128B to 1GB for all the collective communications, significantly reducing the overhead of security-aware operations and achieving bandwidth comparable to the NCCL implementation.

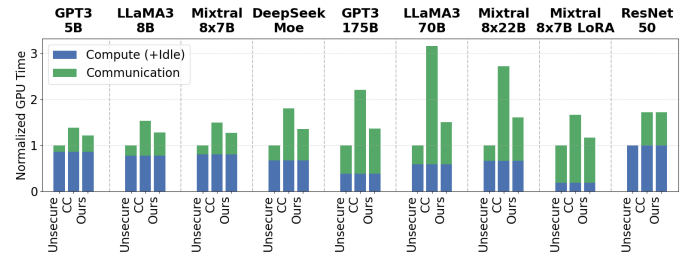


Fig. 13: Normalized GPU time per training epoch.

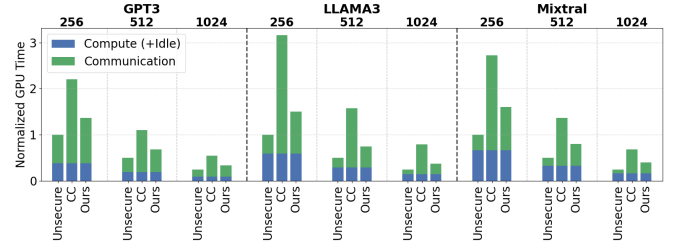


Fig. 14: Normalized GPU time per training epoch (Large).

### C. Characterizing SCALE on ML Workloads

Figure 13 presents normalized GPU time across three configurations. These include the *Unsecure baseline*, *Confidential Computing (CC)* with hardware-based encryption, and our proposed secure system, SCALE. The comparison is evaluated across various LLM workloads by summing up the total Compute and Communication execution time.

As extensively discussed in design section (Section IV), SCALE significantly reduces the communication overhead introduced by secure execution. As show in Figure 13 for the normalized training epoch time, SCALE largely optimizes the exposed communication for both fine-tuning and training across different models (including GPT3, LLaMA3, DeepSeek and etc). For example in LLaMA3 70B training, CC increases the overhead by up to  $6.29\times$  (e.g., LLaMA3 70B), while SCALE reduces it to just  $2.24\times$ , eliminating more than 64% of the secure communication cost. An exception is ResNet-50 (a convolutional neural network), where the reduction in communication time is less significant because of its comparatively low communication volume, as shown in Table IV. As a result, these savings also translate to overall execution time due to the outstanding communication runtime in the flow. Take LLaMA3 70B as an example, When we adjust for the additional communication cost, CC increases total execution time by up to 2.73 times, while SCALE limits this to 1.61 times, recovering over two-thirds of the lost efficiency. Across the board, SCALE can achieve a range of 40-70% reduction in total security overhead compared to CC except for ResNet-50, where CPU-side encryption, limited to 4 GB/s, dominates). While optimizing the CPU-GPU path is out of scope, our method can be combined with complementary solutions [34], [36], [42], [48]. Since compute time remains constant and SCALE only utilizes idle SMs (Section IV-B), the improvement confirms that computation and compute-communication overlap are preserved. SCALE thus ensures secure communication without compromising efficiency.

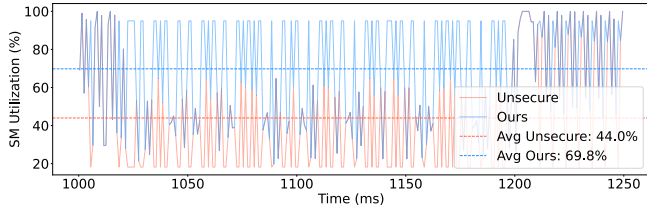


Fig. 15: SM Utilization Captured in LLaMA3 8B.

**Large-scale System Evaluation.** Figure 14 compares training time for *Unsecure*, *CC*, and *SCALE* on a 256, 512 and 1024 GPU cluster. We train GPT3 175B, LLaMA3 70B, and Mixtral 8x22B using hybrid tensor and data parallelism. Starting from  $DP=8$ , we progressively increase the data-parallel degree while fixing the global batch size at 1024. As the  $DP$  degree increases, the per-GPU batch size decreases, shortening per-GPU compute time. Although the amount of data each GPU exchanges remains roughly constant, the overall collective size grows, which in turn increases synchronization latency. Across all models, *CC* increases training time by 2.2–3.2 $\times$ , while *SCALE* restores 50–60 % of that overhead, achieving 1.6–2.1 $\times$  speedup over *CC*.

#### D. Hardware Resource Overheads

**SM Utilization.** Figure 15 shows SM utilization over a 250 ms window (1.00–1.25 s). We conservatively assume that software encryption kernels (e.g., XOR, GHASH) can use up to 90% of available SMs, based on observed peak utilizations near 100%. Secure runs (red) exhibit higher average SM utilization (69.8%) compared to unsecure runs (blue), due to additional encryption activity within communication kernels. The utilization gap between secure and unsecure runs varies depending on the workload; in our setup, it ranges from 13% to 65%.

**Memory.** Memory overhead arises from two sources: (1) GHASH buffers for intermediate authentication metadata, and (2) precomputed OTPs for AES-CTR, stored in on-package HBM. Both are tunable parameters that trade memory for reduced encryption latency. In our evaluation, we allocate 1 GB each for GHASH and OTP buffers—less than 1.5% of the 141 GB available on H200.

**Power Consumption.** We profiled CUDA-based AES-CTR, XOR, and GHASH kernels on both H100 and H200 GPUs. Their power consumption ranged from 92 to 108 W—well below the 700 W TDP—indicating that encryption overhead remains within the GPU’s power budget. Importantly, encryption-related energy is not entirely additive: even without software encryption, hardware paths (e.g., copy engines) would still be active for data encryption. Thus, software encryption reuses existing compute and data transfer paths, resulting in only modest additional energy overhead.

#### E. Sensitivity Study: More Powerful HW Encryption Engines

We sweep the per-link encrypted bandwidth of a hardware-only baseline up to 512 GB/s and compare it with *SCALE*.

**Performance.** Figure 16 shows that *SCALE* maintains a clear performance lead while the unidirectional bandwidth is

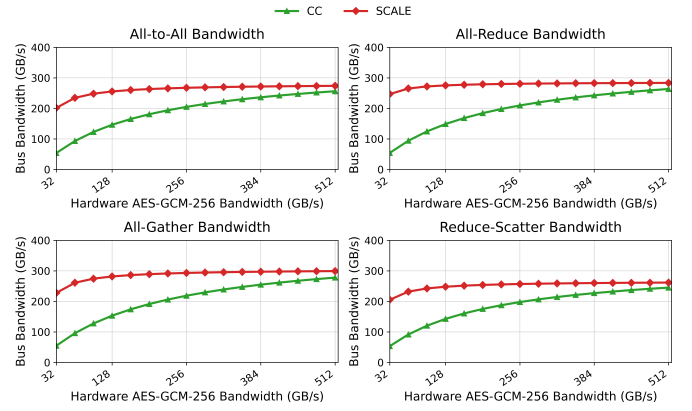


Fig. 16: Ablation Study: AES Hardware Engine Bandwidth

512 GB/s. Beyond that point the hardware-only curve flattens and approaches *SCALE*. *SCALE* also benefits from higher hardware AES-GCM throughput since it can switch between software and hardware paths.

**Area.** Graphcore’s GC200 integrates four Secure Exchange Pipe (SXP) engines that together reach 32 GB/s bidirectional (16 GB/s per direction) and occupy <1 % of an 823 mm<sup>2</sup> die in 7 nm [44], [45]. NVIDIA H100 is a similar size (814 mm<sup>2</sup>, 5 nm). Replicating the same four-engine cluster would therefore cost about 1 % die area. Reaching 512 GB/s unidirectional NVLink would need roughly 113 engines about 28 % die area—before accounting for additional routing.

**Power.** Exact power numbers for NVIDIA’s in-package engines are not public, but a recent Intel 4 nm AES-CTR prototype reports 21 mW at 8.3 Gb/s, i.e. 2.53 pJ/bit  $\approx$  20 pJ/B [20]. Also, the GHASH block that completes AES-GCM is typically  $\lesssim$  10% of the AES core’s dynamic power [13], [24].

$$E_{\text{GCM}} = E_{\text{CTR}} \times 1.10 = 20 \text{ pJ/B} \times 1.10 = 22 \text{ pJ/B}.$$

$$P = \text{Bandwidth} \times E_{\text{GCM}} = 512 \text{ GB/s} \times 22 \text{ pJ/B} \approx 11 \text{ W}.$$

The hardware path consumes roughly 11% the energy of a software-only AES-GCM running on the GPU SMs.

Figure 17 shows the power consumption measured during GPT3 175B training (115.6 s runtime, 215 W average GPU power, 24.9 kJ total energy, and 3.0 TB of per-GPU network traffic). Hardware encryption maintains nearly constant energy per byte (22 pJ/B, accounting for 0.27% of total GPU energy), and the total energy change can be approximated as

$$\Delta \text{Energy} \approx E_{\text{GCM}} \times \text{DataSize} + P_{\text{idle}} \times (\text{Time}_{\text{CC}} - \text{Time}_{\text{base}})$$

where  $P_{\text{idle}} = 77 \text{ W}$  is the idle GPU power measured via *nvidia-smi*. For hardware encryption bandwidths of 64, 128, 256, and 512 GB/s, the power overheads over the unsecure baseline are 42%, 16%, 4.5%, and 4%, respectively. Although *SCALE* consumes 4.7 $\times$  more energy per byte than hardware-only encryption, it still improves end-to-end efficiency, incurring only a 16% overhead over the baseline at 64 GB/s by reducing overall runtime.

**Take-away.** Adding more hardware encryption engines keeps SM occupancy low and can be power-efficient, but it also

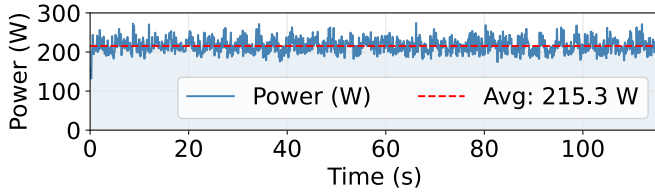


Fig. 17: GPU0 power during one GPT-3 175B training epoch.

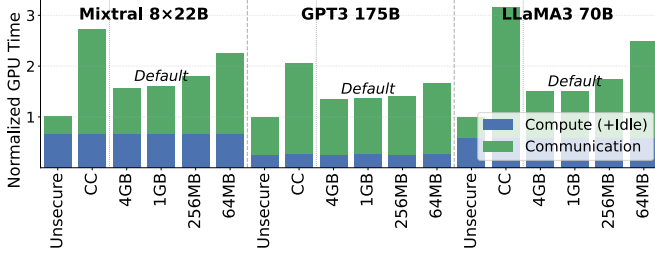


Fig. 18: Normalized GPU time vs. OTP buffer size.

increases die area and total board power. To the best of our knowledge, all commercial GPU, including H100, B200, and pro 6000, do not support such powerful encryption engines.

#### F. Sensitivity Study: Less available resource

**Memory Sensitivity.** Figure 18 shows SCALE with OTP buffers ranging from 4 GB to 64 MB. Runtime increases gradually as the buffer shrinks because smaller buffers trigger more frequent hardware fallback. In practice, a 1 GB buffer captures nearly all benefit, while reducing it to 256 MB offers a practical trade-off for large models that saturate memory and still outperforms CC by 17–21 %.

More importantly, under the same memory budget, SCALE achieves higher performance gains than other techniques. We further conducted experiments where we increased the Distributed Adam optimizer’s gradient chunk size for All-Reduce from 110 MB to 120 MB—a common approach in distributed training to improve efficiency. While this change raised peak memory usage by approximately 8 GB, it only improved performance over the baseline CC by 4.5%. In our study, each GPU’s batch size was already maximized and could not be further increased. Therefore, allocating memory to SCALE yields significantly higher performance returns than using it for larger optimizer buffers or additional batch sizes.

**Idle-Resource Sensitivity.** Figure 19 shows SCALE performance under limited compute slack by reducing the SMs available to encryption from 100 % down to 10 %. As idle resources shrink, overlap between encryption and communication decreases, causing a gradual, near-linear slowdown. At 50%-idle, SCALE remains on average 33 % faster than

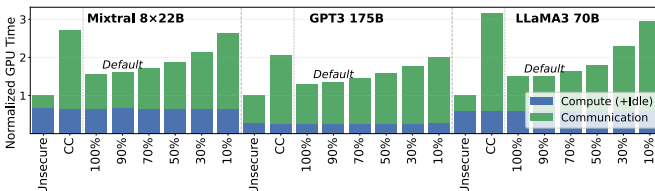


Fig. 19: Normalized GPU time vs. idle resources.

hardware-only CC across all models, while at 10%-idle the gain drops to only 4 %. This shows that SCALE’s benefit decreases smoothly with available resources it can leverage.

**Take-away.** Across both studies, SCALE sustains its benefit under constrained memory and compute resources by opportunistically reusing idle GPU cycles for encryption. Its performance degrades gracefully with less idle time, indicating that compute contention is the primary scalability limit.

## VIII. RELATED WORK

**Confidential Computing for ML.** Recent efforts investigate secure ML under CC. PipeLLM [42] accelerates confidential inference by optimizing key-value cache access, while FastTrack [48] mitigates MAC-serialization bottlenecks in single-GPU AES-GCM. IOValve [21] explores RNIC-based AES-GCM offload to reduce encryption latency in secure distributed ML. SCALE builds on this line of work by enabling secure, topology-aware collectives across multiple GPUs. Unlike prior systems, which either optimize inference or focus on single-GPU or NIC-terminated protection boundaries, SCALE targets large-scale distributed training and performs encryption inside the GPU TEE to protect both inter-node and intra-node communication over NVLink, alleviating the encrypted communication bottleneck in confidential multi-GPU training.

**Non-CC GPU-based TEEs.** Graviton [46] and HIX [16] propose early GPU TEE designs. They focus on securing the control path between CPU and GPU but do not support scalable data movement or encrypted collectives. Na et al. [26] suggest OTP reuse and metadata batching in shared-memory multi-GPU setups. However, their approach assumes bandwidth-equivalent AES hardware and does not address collective communication semantics critical to distributed ML. SCALE is built for realistic, large-scale workloads on commercial CC GPUs, addressing practical performance–security trade-offs overlooked by prior conceptual GPU TEE work.

## IX. CONCLUSION

We present SCALE, a comprehensive study that identifies secure communication as a key bottleneck and examines ways to address it in confidential distributed ML training. SCALE combines hybrid software and hardware encryption, out-of-critical-path authentication, and security-aware collectives to minimize communication overhead. As a result, SCALE reduces communication overhead from up to  $6.29\times$  to  $2.24\times$ , recovering much of the performance lost due to encryption, without sacrificing security guarantees.

## X. ACKNOWLEDGMENT

This work was supported in part by the ACE Center for Evolvable Computing, one of the seven centers of the SRC JUMP 2.0 program. This research also received support from the Future of Life Institute and the Blake Borgeson Foundation. We thank FlexHEGs for their support and insightful discussions. We also acknowledge Prof. Murali Annavaram for his generous support throughout the project.

## REFERENCES

- [1] N. Corporation, "NVLink™ high-speed gpu interconnect: Application performance brief," White paper, 2016, basic overview of NVLink interconnect architecture; does not explicitly describe the retry protocol. [Online]. Available: <https://info.nvidia.com/rs/nvidia/images/NVIDIA%20NVLink%20High-Speed%20Interconnect%20Application%20Performance%20Brief.pdf>
- [2] N. Corporation, "Nvidia hopper architecture technical overview," NVIDIA Corporation, Tech. Rep., March 2022, white Paper. [Online]. Available: <https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth>
- [3] N. Corporation, "Nvidia hopper architecture technical overview," NVIDIA Corporation, Tech. Rep., March 2022, white Paper. [Online]. Available: <https://www.advancedclustering.com/wp-content/uploads/2022/03/gtc22-whitepaper-hopper.pdf>
- [4] N. Corporation, "Optimizing multi-gpu communication with nccl and gputhread async," in *GTC Conference Proceedings*, 2024, presented at GTC (GPU Technology Conference) 2024.
- [5] N. Corporation, *NCCL Documentation: Environment Variables*, <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/env.html>, 2025, accessed: April 11, 2025.
- [6] D. Dai, C. Deng, C. Zhao, R. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu *et al.*, "Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models," *arXiv preprint arXiv:2401.06066*, 2024.
- [7] G. Dhanuskodi, S. Guha, V. Krishnan, A. Manjunatha, M. O'Connor, R. Nertney, and P. Rogers, "Creating the first confidential gpu: The team at nvidia brings confidentiality and integrity to user code and data for accelerated computing," 2023.
- [8] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/counter mode (gcm) and gmac," *NIST Special Publication 800-38D*, 2007.
- [9] E. Feng, D. Du, Y. Xia, and H. Chen, "Efficient distributed secure memory with migratable merkle tree," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 347–360.
- [10] L. Floridi and M. Chiriatti, "Gpt-3: Its nature, scope, limits, and consequences," *Minds and Machines*, vol. 30, pp. 681–694, 2020.
- [11] Google Cloud, "Confidential computing: Secure your workloads with confidential vms," <https://cloud.google.com/security/products/confidential-computing>, 2024, accessed: 2025-04-04.
- [12] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.
- [13] X. Guo and R. Karri, "Low-cost concurrent error detection for gcm and ccm," in *Proc. Field-Programmable Logic and Applications (FPL)*, 2013, table 4: ASIC power overhead 0.3–12.6%.
- [14] R. Haakegaard and J. Lang, "The elliptic curve diffie-hellman (ecdh)," *Online* at <https://koclab.cs.ucsb.edu/teaching/ecc/project/2015Projects/Haakegaard+Lang.pdf>, 2015.
- [15] S. Imambi, K. B. Prakash, and G. Kanagachidambaresan, "Pytorch," *Programming with TensorFlow: solution for edge computing applications*, pp. 87–104, 2021.
- [16] I. Jang, A. Tang, T. Kim, S. Sethumadhavan, and J. Huh, "Heterogeneous isolated execution for commodity gpus," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2019, pp. 455–468.
- [17] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. l. Casas, E. B. Hanna, F. Bressand *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.
- [18] B. Koonce, "Resnet 50," in *Convolutional neural networks with swift for tensorflow: image recognition and dataset categorization*. Springer, 2021, pp. 63–72.
- [19] O. Kuchaiev, J. Li, H. Nguyen, O. Hrinchuk, R. Leary, B. Ginsburg, S. Krizan, S. Beliaev, V. Lavrukhin, J. Cook *et al.*, "Nemo: a toolkit for building ai applications using neural modules," *arXiv preprint arXiv:1909.09577*, 2019.
- [20] R. Kumar, V. B. Suresh, M. A. Anders, S. K. Hsu, A. Agarwal, V. K. De, and S. K. Mathew, "An 8.3-to-18gbps reconfigurable sca-resistant/dual-core/blind-bulk aes engine in intel 4 cmos," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65. IEEE, 2022, pp. 1–3.
- [21] S. Lee, J. Drean, Y. Tan, and M. Peinado, "Iovalve: Leakage-free i/o sandbox for large-scale untrusted data processing," in *Proceedings of the 32nd ACM Conference on Computer and Communications Security (CCS)*, Taipei, Taiwan, Oct. 2025, to appear.
- [22] M. Li, Y. Zhang, Z. Lin, and Y. Solihin, "Exploiting unprotected {I/O} operations in {AMD's} secure encrypted virtualization," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 1257–1272.
- [23] S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan, P. Damania, and S. Chintala, "Pytorch distributed: experiences on accelerating data parallel training," *Proc. VLDB Endow.*, vol. 13, no. 12, p. 3005–3018, Aug. 2020. [Online]. Available: <https://doi.org/10.14778/3415478.3415530>
- [24] D. A. McGrew and J. Viegas, "The Galois/Counter mode of operation (GCM)," NIST Modes of Operation submission, 2005, sec. 6, "In hardware, GCM adds a negligible amount of overhead compared to a pipelined AES implementation."
- [25] Microsoft, "Gpu options for azure confidential computing," <https://learn.microsoft.com/en-us/azure/confidential-computing/gpu-options>, 2024, accessed: 2025-04-04.
- [26] S. Na, J. Kim, S. Lee, and J. Huh, "Supporting secure multi-gpu computing with dynamic and batched metadata management," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 204–217.
- [27] NVIDIA, "NVIDIA Hopper Architecture In-Depth," <https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/>, 2022, accessed: 2025-08-01.
- [28] NVIDIA, "Nvidia linux open gpu kernel module source," <https://github.com/NVIDIA/open-gpu-kernel-modules>, 2022, accessed: 2025-04-09.
- [29] NVIDIA, "Confidential computing on h100 gpus for secure and trustworthy ai," <https://developer.nvidia.com/blog/confidential-computing-on-h100-gpus-for-secure-and-trustworthy-ai/>, 2023, accessed: 2025-07-31.
- [30] NVIDIA, "Announcing nvidia secure ai general availability," <https://developer.nvidia.com/blog/announcing-nvidia-secure-ai-general-availability/>, 2024, accessed: 2025-07-31.
- [31] NVIDIA, (2024) Megatron distributed checkpoint. Accessed: 2025-07-28. [Online]. Available: [https://docs.nvidia.com/megatron-core/developer-guide/latest/api-guide/dist\\_checkpointing.html](https://docs.nvidia.com/megatron-core/developer-guide/latest/api-guide/dist_checkpointing.html)
- [32] NVIDIA, "NCCL (NVIDIA Collective Communications Library)," <https://github.com/NVIDIA/nccl>, 2024, accessed: 2025-04-11.
- [33] NVIDIA Corporation, "Nvidia blackwell architecture: Advancing the era of generative ai," <https://resources.nvidia.com/en-us-blackwell-architecture>, 2024, accessed: 2025-10-19.
- [34] NVIDIA Corporation, "Nvidia trusted computing solutions release notes," <https://docs.nvidia.com/570TRD1-trusted-computing-solutions-release-notes.pdf>, NVIDIA Corporation, Release Notes RN-11468-001\_570\_03, Mar. 2025.
- [35] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "Zero: Memory optimizations toward training trillion parameter models," 2020. [Online]. Available: <https://arxiv.org/abs/1910.02054>
- [36] E. Sakata, "Announcing NVIDIA Secure AI General Availability," <https://developer.nvidia.com/blog/announcing-nvidia-secure-ai-general-availability/>, Apr. 2025, nVIDIA Technical Blog.
- [37] W. Shi, H.-H. Lee, M. Ghosh, and C. Lu, "Architectural support for high speed protection of memory integrity and confidentiality in multiprocessor systems," in *Proceedings. 13th International Conference on Parallel Architecture and Compilation Techniques, 2004. PACT 2004.*, 2004, pp. 123–134.
- [38] W. Shi, H. Lee, M. Ghosh, C. Lu, and A. Boldyreva, "High efficiency counter mode security architecture via prediction and precomputation," in *32nd International Symposium on Computer Architecture (ISCA'05)*, 2005.
- [39] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," *arXiv preprint arXiv:1909.08053*, 2019.
- [40] S. Sridharan, T. Heo, L. Feng, Z. Wang, M. Bergeron, W. Fu, S. Zheng, B. Coutinho, S. Rashidi, C. Man, and T. Krishna, "Chakra: Advancing performance benchmarking and co-design using standardized execution traces," *arXiv preprint*, vol. arXiv:2305.14516, 2023, arXiv:2305.14516 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2305.14516>
- [41] G. E. Suh, C. W. O'Donnell, and S. Devadas, "Aegis: A single-chip secure processor," *IEEE Design & Test of Computers*, vol. 24, no. 6, pp. 570–580, 2007.

- [42] Y. Tan, C. Tan, Z. Mi, and H. Chen, "Pipelml: Fast and confidential large language model services with speculative pipelined encryption," *arXiv preprint arXiv:2411.03357*, 2024.
- [43] N. N. Team, "Issue #689: Nccl channel allocation discussion," <https://github.com/NVIDIA/nccl/issues/689>, 2021, accessed: April 11, 2025.
- [44] N. Toon, "Introducing 2nd generation ipu systems for ai at scale," <https://www.graphcore.ai/posts/introducing-second-generation-ipu-systems-for-ai-at-scale>, Jul. 2020, graphcore Blog, accessed 2025-06-20.
- [45] K. Vaswani, S. Volos, C. Fournet, A. N. Diaz, K. Gordon, B. Vembu, S. Webster, D. Chisnall, S. Kulkarni, G. Cunningham *et al.*, "Confidential computing within an {AI} accelerator," in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, 2023.
- [46] S. Volos, K. Vaswani, and R. Bruno, "Graviton: Trusted execution environments on {GPUs}," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018, pp. 681–696.
- [47] B. Wan, M. Han, Y. Sheng, Y. Peng, H. Lin, M. Zhang, Z. Lai, M. Yu, J. Zhang, Z. Song, X. Liu, and C. Wu, "Bytecheckpoint: A unified checkpointing system for large foundation model development," 2025. [Online]. Available: <https://arxiv.org/abs/2407.20143>
- [48] Y. Wang, R. Rajat, J. Lee, T. Tang, and M. Annavaram, "Fastrack: Fast io for secure ml using gpu tees," 2024. [Online]. Available: <https://arxiv.org/abs/2410.15240>
- [49] W. Won, T. Heo, S. Rashidi, S. Sridharan, S. Srinivasan, and T. Krishna, "Astra-sim2. 0: Modeling hierarchical networks and disaggregated systems for large-model training at scale," in *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2023, pp. 283–294.
- [50] C. Yan, D. Engländer, M. Prvulovic, B. Rogers, and Y. Solihin, "Improving cost, performance, and security of memory encryption and authentication," in *33rd International Symposium on Computer Architecture (ISCA'06)*, 2006, pp. 179–190.